HRCMS Database Reference

Relational schema, primary and foreign keys, data types, constraints, and database logic

Project: Hostel Room Management and Complaint System (HRCMS)
Database: local MySQL database `hrsm`
Schema model: 13 relational tables, 4 views, 15 triggers, and 5 stored procedures
Prepared: 26 October 2026

Current database snapshot

Block	Active students	Rooms	Active workers
A Block	50	16	5
B Block	50	14	5
G Block	50	15	5

Account totals: 170 user records: 150 active students, 15 active workers, 1 disabled worker, 3 wardens, and 1 administrator. The database also contains 151 room-allocation records (including historical allocation data), 11 complaints, 6 complaint categories, 4 room types, and 45 rooms.

What this reference covers

Use this document as a data dictionary and viva reference. It documents the schema declared in the project SQL files. MySQL BOOLEAN is an alias for TINYINT(1); ENUM columns restrict values to the listed alternatives. A foreign key (FK) references a primary or unique key in another table.

Application flow

```
React/Vite web interface -> Express API -> MySQL database
```

The browser communicates with the backend API. The backend owns database credentials, validates requests, runs parameterized SQL, and calls stored procedures inside transactions.

1. Relational model and relationships

The database separates shared identity from role-specific details, keeps room history as an allocation relation, and records complaint workflow and audit history in separate relations.

Parent	Related table	Cardinality	Interpretation
users	user_phone	1 to many	One user can have zero or multiple phone rows.
users	student / warden / maintenance_staff	1 to 0..1 per subtype	Role-specific detail rows use the user ID as both PK and FK; role guards verify the subtype. ADMIN needs only users.
warden	hostel_block	1 to 0..1; block side 0..1	hostel_block.warden_id is nullable and UNIQUE, so one block has at most one warden and one warden manages at most one block.
hostel_block	room	1 to many	Each room belongs to one block.
room_type	room	1 to many	Each room has one type, which provides its capacity and annual fee.
student	room_allocation	1 to many over time	Each allocation belongs to one student. A unique generated active flag limits a student to one active allocation.
room	room_allocation	1 to many over time	Each allocation names one room. The allocation table preserves vacated history.
hostel_block / complaint_category	maintenance_staff	1 to many each	Each staff profile belongs to one block and one specialty; a block/category can have multiple staff.
student / room / complaint_category	complaint	1 to many each	Each complaint belongs to one student, one room, and one category.
maintenance_staff	complaint	1 to many; optional assignment	A complaint may be unassigned; an assigned staff member can handle multiple complaints.
complaint	complaint_log	1 to many	Every workflow change can be recorded as an audit row.
complaint	feedback	1 to 0..1	feedback.complaint_id is both its PK and FK, allowing one feedback row per complaint.

Simplified key map

```
users(user_id) -> student(student_id), warden(warden_id), maintenance_staff(staff_id), user_phone(user_id)
hostel_block(block_id) -> room(block_id), maintenance_staff(block_id); room_type(type_code) -> room(type_code)
student(student_id) + room(room_id) -> room_allocation(student_id, room_id)
student(student_id) + room(room_id) + complaint_category(category_id) + maintenance_staff(staff_id) -> complaint
complaint(complaint_id) -> complaint_log(complaint_id), feedback(complaint_id)
```

Arrow means “the left-side key is referenced by a foreign key on the right-side table.”

2. Data dictionary: identity and accounts

users

Shared identity, authentication, role, and activation state.

Column	MySQL type	Null / default	Key / reference	Meaning
user_id	INT AUTO_INCREMENT	NOT NULL	PK	Unique account identifier.
full_name	VARCHAR(100)	NOT NULL	—	Person or administrator display name.
email	VARCHAR(120)	NOT NULL	UNIQUE	Login email; unique across all accounts.
password_hash	CHAR(60)	NOT NULL	—	bcrypt password hash, not plaintext password.
role	ENUM('STUDENT','WARDEN','STAFF','ADMIN')	NOT NULL	—	Account role; immutable after creation by trigger.
is_active	BOOLEAN	NOT NULL; DEFAULT TRUE	—	Soft activation flag; disabled accounts retain history.
created_at	TIMESTAMP	NOT NULL; DEFAULT CURRENT_TIMESTAMP	—	Account creation time.

Rules: Unique email. Role-specific subtype rows are validated by triggers.

user_phone

Multivalued phone numbers stored as one phone number per row.

Column	MySQL type	Null / default	Key / reference	Meaning
user_id	INT	NOT NULL	PK part; FK -> users.user_id	Owner of the phone number.
phone_no	VARCHAR(15)	NOT NULL	PK part	One phone number.

Rules: Composite primary key (user_id, phone_no); phone format checked by chk_phone; deleting the user cascades to phone rows.

student

Student-specific registration and academic data.

Column	MySQL type	Null / default	Key / reference	Meaning
student_id	INT	NOT NULL	PK; FK -> users.user_id	Same identifier as the shared user account.
reg_no	VARCHAR(15)	NOT NULL	UNIQUE	Student registration number.
gender	ENUM('MALE','FEMALE')	NOT NULL	—	Used to validate block compatibility.
department	VARCHAR(60)	NOT NULL	—	Student’s academic department.
year_of_study	TINYINT	NOT NULL	—	Study year; chk_year restricts it to 1..5.
guardian_name	VARCHAR(100)	NULL	—	Optional guardian name.
guardian_phone	VARCHAR(15)	NULL	—	Optional guardian phone in the physical schema.

Rules: Deleting the user cascades to the student row. The role guard requires users.role = STUDENT.

2. Data dictionary: wardens and workers

warden

Warden-specific information; the linked user contains shared login details.

Column	MySQL type	Null / default	Key / reference	Meaning
warden_id	INT	NOT NULL	PK; FK -> users.user_id	Same identifier as the warden's user account.
designation	VARCHAR(50)	NOT NULL; DEFAULT Warden	—	Warden title or designation.
office_room	VARCHAR(20)	NULL	—	Optional office location.

Rules: Deleting the user cascades to the warden row. Role guard requires users.role = WARDEN.

maintenance_staff

Worker specialty, assigned block, and shift.

Column	MySQL type	Null / default	Key / reference	Meaning
staff_id	INT	NOT NULL	PK; FK -> users.user_id	Same identifier as the worker's account.
category_id	INT	NOT NULL	FK -> complaint_category.category_id	Worker specialty/service domain.
block_id	INT	NOT NULL	FK -> hostel_block.block_id	Block the worker is assigned to serve.
shift	ENUM('MORNING','EVENING','NIGHT')	NOT NULL; DEFAULT MORNING	—	Worker shift.

Rules: Role guard requires users.role = STAFF. Complaint assignment guards require specialty and block to match.

Role design note

`users` stores fields common to all roles. `student`, `warden`, and `maintenance_staff` are subtype tables whose primary keys are also foreign keys to `users`. This avoids repeating email, password hash, name, and active state in each role table.

2. Data dictionary: hostel layout

hostel_block

Hostel block identity, gender category, floors, and managing warden.

Column	MySQL type	Null / default	Key / reference	Meaning
block_id	INT AUTO_INCREMENT	NOT NULL	PK	Unique block identifier.
block_name	VARCHAR(30)	NOT NULL	UNIQUE	Block name.
gender_type	ENUM('MALE','FEMALE')	NOT NULL	—	Permitted resident gender for the block.
total_floors	TINYINT	NOT NULL	—	Number of floors; chk_floors allows 1..20.
warden_id	INT	NULL	UNIQUE; FK -> warden.warden_id	Optional managing warden; unique across blocks.
is_active	BOOLEAN	NOT NULL; DEFAULT TRUE	—	Soft activation flag.

Rules: Deleting a block's warden sets warden_id to NULL. Deleting a block cascades to its rooms.

room_type

Reusable room capacity and annual fee catalog.

Column	MySQL type	Null / default	Key / reference	Meaning
type_code	VARCHAR(10)	NOT NULL	PK	Type code, e.g. SINGLE or QUAD.
capacity	TINYINT	NOT NULL	—	Maximum number of residents; chk_capacity allows 1..6.
annual_fee	DECIMAL(10,2)	NOT NULL	—	Annual room fee; chk_fee requires a non-negative amount.
is_active	BOOLEAN	NOT NULL; DEFAULT TRUE	—	Whether the type can be selected.

Rules: Updating type_code cascades to rooms.

room

Physical room in a block, with its type and availability state.

Column	MySQL type	Null / default	Key / reference	Meaning
room_id	INT AUTO_INCREMENT	NOT NULL	PK	Unique room identifier.
block_id	INT	NOT NULL	FK -> hostel_block.block_id	Containing block.
room_number	VARCHAR(10)	NOT NULL	UNIQUE KEY uq_room with block_id	Room label within its block.
floor_no	TINYINT	NOT NULL	—	Zero-based floor number; chk_floor requires >= 0.
type_code	VARCHAR(10)	NOT NULL	FK -> room_type.type_code	Capacity/fee type.
has_ac	BOOLEAN	NOT NULL; DEFAULT FALSE	—	Whether the room has air conditioning.
status	ENUM('ACTIVE','UNDER_MAINTENANCE')	NOT NULL; DEFAULT ACTIVE	—	Operational status.
is_active	BOOLEAN	NOT NULL; DEFAULT TRUE	—	Administrative activation state.

Rules: Room number is unique only within a block. New active allocations are blocked for inactive or maintenance rooms.

2. Data dictionary: room history

room_allocation

Associative and historical relation between students and rooms.

Column	MySQL type	Null / default	Key / reference	Meaning
allocation_id	INT AUTO_INCREMENT	NOT NULL	PK	Unique allocation event.
student_id	INT	NOT NULL	FK -> student.student_id	Student receiving the room.
room_id	INT	NOT NULL	FK -> room.room_id	Allocated room.
academic_year	CHAR(7)	NOT NULL	—	Academic year in YYYY-YY format.
allocated_on	DATE	NOT NULL	—	Allocation start date.
vacated_on	DATE	NULL	—	Optional date the student vacated.
status	ENUM('ACTIVE','VACATED')	NOT NULL; DEFAULT ACTIVE	—	Whether this allocation is current or historical.
active_flag	TINYINT GENERATED STORED	Generated: ACTIVE -> 1; otherwise NULL	UNIQUE with student_id	Generated helper used to enforce one active room per student.

Rules: uq_one_active_room(student_id, active_flag) permits only one ACTIVE row per student; VACATED rows yield NULL and can repeat. chk_dates validates date order; chk_academic_year validates the year format. idx_allocation_room(room_id, status) supports room occupancy lookups. Deleting a student cascades to allocation history.

Allocation safeguards

The stored procedure locks the room and student rows before inserting. A trigger then checks room availability, room capacity, student account validity, and gender compatibility. This combines transaction-level locking with database-level validation.

Example key

A room_allocation row has its own allocation_id primary key. student_id and room_id are foreign keys. The generated active_flag plus the unique pair (student_id, active_flag) enforces the “one active room per student” rule while retaining any number of past allocations.

2. Data dictionary: complaint domain

complaint_category

Lookup table for service domain, default priority, and service target.

Column	MySQL type	Null / default	Key / reference	Meaning
category_id	INT AUTO_INCREMENT	NOT NULL	PK	Unique service category identifier.
category_name	VARCHAR(40)	NOT NULL	UNIQUE	Category such as electrical, plumbing, or Wi-Fi/network.
default_priority	ENUM('LOW','MEDIUM','HIGH','URGENT')	NOT NULL; DEFAULT MEDIUM	—	Default priority for new complaints.
sla_hours	SMALLINT	NOT NULL	—	Target response/completion duration in hours; chk_sla > 0.
is_active	BOOLEAN	NOT NULL; DEFAULT TRUE	—	Whether the category can receive new complaints.

complaint

Student-reported maintenance issue and its current workflow state.

Column	MySQL type	Null / default	Key / reference	Meaning
complaint_id	INT AUTO_INCREMENT	NOT NULL	PK	Unique complaint identifier.
student_id	INT	NOT NULL	FK -> student.student_id	Student who raised the complaint.
room_id	INT	NOT NULL	FK -> room.room_id	Room associated with the complaint.
category_id	INT	NOT NULL	FK -> complaint_category.category_id	Service domain.
title	VARCHAR(120)	NOT NULL	—	Short issue title; chk_title requires at least 5 characters.
description	TEXT	NOT NULL	—	Full issue description.
priority	ENUM('LOW','MEDIUM','HIGH','URGENT')	NOT NULL	—	Complaint urgency.
status	ENUM('OPEN','ASSIGNED','IN_PROGRESS','RESOLVED','CLOSED','REJECTED')	NOT NULL; DEFAULT OPEN	—	Workflow state.
assigned_staff_id	INT	NULL	FK -> maintenance_staff.staff_id	Current assigned worker, if any.
created_at	TIMESTAMP	NOT NULL; DEFAULT CURRENT_TIMESTAMP	—	Creation timestamp.
updated_at	TIMESTAMP	NOT NULL; DEFAULT CURRENT_TIMESTAMP ON UPDATE	—	Automatically updated when row changes.
resolved_at	TIMESTAMP	NULL	—	Set when complaint is resolved; cleared on reopen.

Rules: Deleting the assigned worker sets assigned_staff_id to NULL; other principal links are retained by default/restrict behavior. idx_complaint_status(status) supports status filtering. idx_complaint_room(room_id, category_id, status) supports duplicate-active-complaint checks. Guards require the complaint room to be the student’s active room and validate staff block/category on assignment.

2. Data dictionary: history and feedback

complaint_log

Audit trail of complaint status and assignment changes.

Column	MySQL type	Null / default	Key / reference	Meaning
log_id	INT AUTO_INCREMENT	NOT NULL	PK	Unique audit event.
complaint_id	INT	NOT NULL	FK -> complaint.complaint_id	Complaint being audited.
old_status	VARCHAR(15)	NULL	—	Previous status, if there was one.
new_status	VARCHAR(15)	NOT NULL	—	Status after the event.
changed_by	INT	NULL	FK -> users.user_id	Actor who caused the change.
remark	VARCHAR(255)	NULL	—	Explanation or event note.
changed_at	TIMESTAMP	NOT NULL; DEFAULT CURRENT_TIMESTAMP	—	Time of event.

Rules: Complaint insert/update triggers create log rows. Deleting a complaint cascades to its logs; deleting an actor sets changed_by to NULL.

feedback

One rating and optional comment for a complaint.

Column	MySQL type	Null / default	Key / reference	Meaning
complaint_id	INT	NOT NULL	PK; FK -> complaint.complaint_id	Also makes the relation one feedback row per complaint.
rating	TINYINT	NOT NULL	—	Score 1..5 enforced by chk_rating.
comments	VARCHAR(500)	NULL	—	Optional student feedback.
submitted_at	TIMESTAMP	NOT NULL; DEFAULT CURRENT_TIMESTAMP	—	Feedback submission time.

Rules: Feedback is allowed only for a resolved complaint and only for its owner. Inserting feedback automatically closes the complaint. Deleting the complaint cascades to feedback.

Example workflow

```
OPEN -> ASSIGNED -> IN_PROGRESS -> RESOLVED -> CLOSED (when feedback is submitted)
OPEN or ASSIGNED -> REJECTED
RESOLVED -> OPEN (student reopens; assignment is cleared)
```

3. Keys, constraints, and indexes

Primary keys identify rows. Foreign keys preserve relationships. Unique keys prevent duplicate business identifiers, while CHECK constraints validate values at the database boundary.

Table	Key definitions	Reason / effect
users	PRIMARY KEY(user_id); UNIQUE(email)	One account ID; no duplicate login email.
user_phone	PRIMARY KEY(user_id, phone_no)	A user cannot have the same phone number twice.
hostel_block	PRIMARY KEY(block_id); UNIQUE(block_name); UNIQUE(warden_id)	Unique block identity/name and at most one block per warden.
room_type	PRIMARY KEY(type_code)	Stable code referenced by room.
room	PRIMARY KEY(room_id); UNIQUE(block_id, room_number)	Room number may repeat in a different block.
student	PRIMARY KEY(student_id); UNIQUE(reg_no)	One student subtype per user ID and unique registration number.
complaint_category	PRIMARY KEY(category_id); UNIQUE(category_name)	Unique service domain.
maintenance_staff	PRIMARY KEY(staff_id)	One worker profile per staff user.
room_allocation	PRIMARY KEY(allocation_id); UNIQUE(student_id, active_flag)	At most one active allocation per student; history retained.
complaint	PRIMARY KEY(complaint_id)	Unique complaint identity.
complaint_log	PRIMARY KEY(log_id)	Unique audit event.
feedback	PRIMARY KEY(complaint_id)	At most one feedback row per complaint.

CHECK constraints

Constraint	Validation
chk_phone	Phone string matches permitted digit/plus pattern.
chk_floors	total_floors is between 1 and 20.
chk_capacity	room_type.capacity is between 1 and 6.
chk_fee	annual_fee is non-negative.
chk_floor	room.floor_no is non-negative; admin validation also checks against the block floor count.
chk_year	student.year_of_study is between 1 and 5.
chk_sla	complaint_category.sla_hours is greater than zero.
chk_dates	vacated_on is NULL or not earlier than allocated_on.
chk_academic_year	Academic year follows YYYY-YY pattern.
chk_title	Complaint title has at least five characters.
chk_rating	Feedback rating is between 1 and 5.

Named secondary indexes

Index	Columns	Use
idx_allocation_room	room_id, status	Room occupancy and active allocation lookups.
idx_complaint_status	status	Filter complaints by workflow state.
idx_complaint_room	room_id, category_id, status	Check for another active complaint of the same category in a room.
uq_room	block_id, room_number	Enforce room-number uniqueness within each block.
uq_one_active_room	student_id, active_flag	Enforce only one active room per student.

Delete/update actions

CASCADE is used for dependent role details, phone rows, student allocations, and complaint logs/feedback. SET NULL preserves block or complaint records if the optional warden, assigned worker, or actor account is removed. Other foreign keys use MySQL’s default restrictive behavior unless explicitly stated.

4. Views and triggers

Views provide reusable read models. Triggers protect invariants and write audit history when rows change.

View	Purpose / derived data
v_room_occupancy	Room + block + room type + active allocations. Exposes capacity, occupied count, vacancies, and BLOCKED/FULL/AVAILABLE state.
v_complaint_dashboard	Complaint joined to student identity, room/block, category, and assigned worker; calculates age_hours and is_overdue against category SLA.
v_overdue_complaints	Rows from v_complaint_dashboard where is_overdue = 1.
v_staff_workload	Staff identity, specialty, block, active job count, and completed job count.

Trigger catalog

Trigger	Timing	Database rule
trg_allocation_before_insert	Before room allocation insert	Checks active room/block/type, valid active student, gender match, and capacity.
trg_complaint_before_update	Before complaint update	Enforces allowed status transitions; sets/clears resolved_at and assignment on reopen.
trg_complaint_after_insert	After complaint insert	Adds initial complaint_log row.
trg_complaint_after_update	After complaint update	Logs status or assigned-worker changes.
trg_feedback_before_insert	Before feedback insert	Requires resolved complaint and owner actor.
trg_feedback_after_insert	After feedback insert	Closes complaint after feedback; history trigger records transition.
trg_users_role_immutable	Before users update	Prevents role changes after account creation.
trg_student_role_before_insert/update	Before student insert/update	Requires referenced user to have STUDENT role.
trg_warden_role_before_insert/update	Before warden insert/update	Requires referenced user to have WARDEN role.
trg_staff_role_before_insert/update	Before maintenance_staff insert/update	Requires referenced user to have STAFF role.
trg_complaint_owner_before_insert	Before complaint insert	Requires complaint room to be the active room of the active student.
trg_complaint_assignment_before_update	Before complaint update	Assigned worker must be active and match complaint block and category; assigned states require a worker.

Audit actor context

The API sets MySQL session variables @app_user_id and @app_remark before workflow operations. Triggers use these values to attribute history entries. The API clears them before returning a connection to the pool.

5. Stored procedures and transaction behavior

Procedures package multi-step business operations into database routines. The API calls mutations inside explicit transactions so row locks remain held until COMMIT or ROLLBACK.

Procedure	Behavior / validation
sp_allocate_room(student, room, year, actor)	Locks student and room; verifies the actor manages the block and the student has no active room; inserts allocation. Allocation trigger checks capacity and gender.
sp_vacate_room(allocation, actor)	Locks the room; checks block manager; marks an active allocation VACATED and records vacated_on.
sp_raise_complaint(student, category, title, description, priority)	Requires active room and active category; prevents duplicate active issue for same room/category; inserts complaint and initializes audit context.
sp_assign_complaint(complaint, staff, actor)	Allows assignment only for open/assigned complaint; authorizes admin or block warden; worker must be active with matching block and specialty.
sp_update_status(complaint, status, actor, remark)	Checks actor role/ownership, allowed workflow action, and a meaningful remark; updates status. Database trigger enforces transition rules.

Why row locks matter

Two administrators could otherwise read the same last free bed at the same time and both allocate it. The procedure locks the student/room records before checking and inserting. The trigger and unique generated-key rule add additional protection. The API uses READ COMMITTED transactions and retries deadlocks up to three times.

Database setup order

1. Create/select an empty database.
2. Run database/01_schema.sql to create the 13 tables.
3. Run database/02_logic.sql to create views, triggers, and procedures.
4. Run database/03_integrity_guards.sql to create cross-table role/ownership guards.
5. Run the seed script to insert demonstration data through application routines.

Setup scripts contain no DROP statements. The seed script intentionally stops when sample data already exists.

6. Design notes and viva answers

Normal form	HRCMS example
1NF	Each field is atomic; multiple phone numbers are separate user_phone rows instead of a list in users.
2NF	Non-key columns depend on the whole key. For single-column keys this is automatic; user_phone has a composite key and no non-key attributes.
3NF	Non-key values are kept with their own entity: room capacity/fee in room_type, block gender/name in hostel_block, and role details in subtype tables.

Key design decisions

Decision	Why it is used
Shared identity and role subtypes	Keeps login fields once while preserving role-specific student, warden, and worker attributes.
room_allocation history	Separates the student-room relationship from both entities and records dates/status for each stay.
Soft deactivation	is_active flags disable accounts/rooms/categories without deleting records needed for audit or history.
Generated active_flag	Makes a conditional rule (one active allocation per student) enforceable using a unique index.
Complaint audit log	Stores who changed a complaint, what status changed, when it changed, and an optional remark.
Worker assignment guard	Requires the assigned worker's specialty and block to match the complaint.

Short viva answer: explain the schema

"The database has 13 tables. users stores common account data, and student, warden, and maintenance_staff store role-specific data. hostel_block, room_type, and room model hostel inventory. room_allocation records room history and enforces one active room per student. complaint_category, complaint, complaint_log, and feedback support complaint workflow and audit. Primary and foreign keys preserve relationships; checks, unique indexes, triggers, and procedures enforce business rules."

If asked about current scale

"The local demo database currently contains 50 active students in each of A, B, and G Blocks; 15 active workers, five per block; one active warden per block; 45 rooms; and complaint, allocation, and audit data for demonstration."

Physical schema note

The SQL schema still defines optional guardian_name and guardian_phone columns in student. A UI may omit those inputs while the database columns remain nullable. Removing a form field does not itself drop a database column.

Source of truth: project SQL schema, view/logic script, integrity-guard script, and database README. Where runtime data counts are shown, they were read from the local hrsm database on 26 October 2026.